

PROCEDURES IN ASSEMBLY LANGUAGE

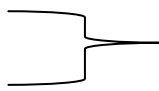
Implementation of procedures using CALL instruction in Intel 8086/88 Assembly language.

I. Procedure and its Implementation

Procedure is a part of code that can be called from your program in order to make some specific task. Procedures make program more structural and easier to understand. Generally procedure returns to the same point from where it was called.

The syntax for procedure declaration is given below.

```

name PROC
    
    ; here goes the code
    ; of the procedure ...
RET
name ENDP
  
```

Here;

name - is the procedure name, the same name should be used to start and end a procedure. This is necessary to check correct closing of procedures.

RET - is the instruction that is used to “return” back the control from the procedure to operating system. The operating system sees the program as a special procedure similar to a “functions” in C language and “methods” in Java.

PROC and **ENDP** - are assembler directives. They are not assembled into any real machine code. Assembler just remembers the address of procedure.

CALL - is the instruction that is used to call a procedure.

Here is an example:

```

MAIN PROC
CALL  m1           ;m1 is the procedure
MOV  AH, 4CH
INT  21H
MAIN ENDP
;Procedure
m1  PROC
MOV  BX, 5
RET           ; return to caller.
m1  ENDP
END MAIN
  
```

The above example calls procedure **m1**, which move constant 5 in the register BX. The procedure then returns the control to the main procedure to execute the next instruction i.e., **MOV AH, 4CH**.

There are several ways to pass parameters to procedure, the easiest way to pass parameters is by using registers. Here is another example of a procedure that receives two parameters

in **AL** and **BL** registers, multiplies these parameters and returns the result in **AX** register:

```
MAIN PROC
MOV  AL, 1
MOV  BL, 2
CALL m2           ; AX = 1 * 2 = 2 and AL = 2
CALL m2           ; AX = 2 * 2 = 4 and AL = 4
CALL m2           ; AX = 4 * 2 = 8 and AL = 8
CALL m2           ; AX = 8 * 2 = 16 and AL = 16
MAIN ENDP

;Procedure
m2  PROC
MUL  BL           ; AX = AL * BL.
RET                ; return to caller.
m2  ENDP
END MAIN
```

In the above example value of **AL** register is updated every time the procedure is called, **BL** register stays unchanged, so this algorithm calculates 2 to the power of 4, with the final result as 16 (or 10h) in **AX** register.